

BERRY GOOD FARMS

Smart BERRY

Rapport d'exécution — Migration & modularisation

Extraction non-régression du monolithe `public/app.jsx`

68 977 lignes → 334 modules ES répartis en 12 modules métier

Document technique destiné au client · Version 1.0

Référence : Plan de Migration & Modernisation v1.1 du 21/08/2026

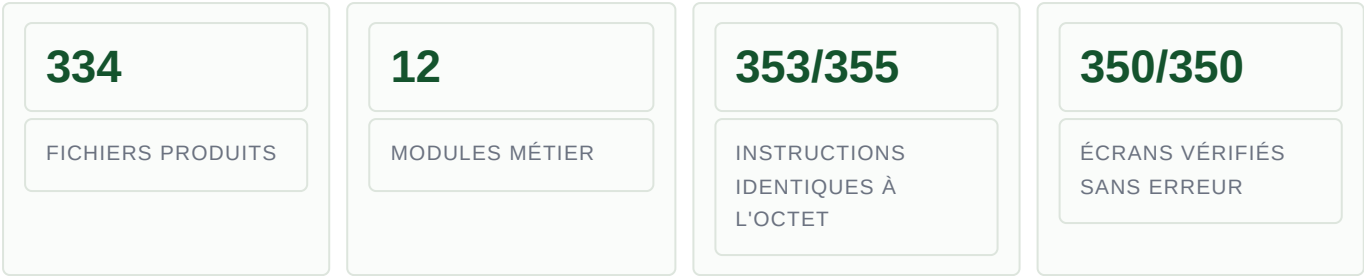
Périmètre : non-régression — aucune modification d'interface ni de logique métier

Sommaire

1. **Résumé exécutif** — ce qui a été livré, en une page
2. **Point de départ** — l'état du dépôt à la prise en main
3. **Principe retenu** — extraction mécanique plutôt que réécriture
4. **Comment les modules sont créés** — la chaîne d'outillage, étape par étape
5. **La structure produite** — les 12 modules métier en détail
6. **Anatomie d'un fichier migré** — exemple commenté
7. **Cas particuliers traités** — les sept pièges rencontrés
8. **Vérification de fidélité** — 353 instructions sur 355 identiques à l'octet
9. **Vérification fonctionnelle** — 350 écrans, 20 profils, parité complète
10. **Déploiement de préversion** — ce qui est en ligne et comment
11. **Ce qui n'a pas changé** — les garanties
12. **Points ouverts** — ce qui reste à décider ou à chiffrer
13. **Reproductibilité** — commandes et outillage livrés
14. **Annexe A** — journal des livraisons

1. Résumé exécutif

L'application Smart BERRY tenait dans un fichier unique de **68 977 lignes** (`public/app.jsx`, 5,6 Mo), compilé en un bundle de 4,0 Mo. Ce fichier a été découpé en **334 modules ES** organisés en **12 modules métier**, sans qu'aucune ligne de logique ni aucun écran ne soit modifié.



Ce que le client obtient

Sujet	Avant	Après
Code source frontend	1 fichier, 68 977 lignes	334 fichiers , 12 modules métier
Fichier le plus long	68 977 lignes	3 153 lignes (<code>QuinzaineTab</code>)
Bundle livré	3 998 437 octets	3 011 995 octets (−24,7 %)
Interface utilisateur	—	strictement identique
Logique métier	—	strictement identique
Reproductibilité	—	outillage livré, régénérable en une commande

La garantie centrale

Le découpage n'est pas une réécriture. Chaque déclaration a été **découpée à l'octet près** depuis le fichier d'origine par un outil automatisé, puis recollée dans son propre fichier. Sur les 355 instructions de premier niveau du monolithe, **353 sont rigoureusement identiques** ; les 2 écarts restants sont documentés, expliqués et sémantiquement équivalents (section 8).

Cette propriété est **mesurée automatiquement**, pas affirmée : la commande `npm run migrate:verify` recompare l'intégralité des fichiers produits au fichier d'origine et échoue si un seul octet diverge.

En une phrase — l'application fait exactement la même chose qu'avant, à l'écran comme dans les calculs ; seule son organisation interne a changé, et ce changement est vérifiable automatiquement à tout moment.

2. Point de départ — l'état du dépôt à la prise en main

Avant d'écrire la moindre ligne, l'état réel du dépôt a été audité. Une arborescence `src/` existait déjà sur la branche de travail et se présentait comme la migration. L'analyse a montré qu'il s'agissait en réalité d'une **réécriture indépendante**, sans rapport avec l'application existante.

Vérification	Monolithe <code>app.jsx</code>	Arborescence <code>src/</code> trouvée
Composants « Tab » définis	120	0
Pages nommées au plan (39 contrôlées)	37 présentes	0 présente
Couche de données	Firebase / Firestore	Supabase PostgreSQL
Vues réellement câblées	120 onglets, 20 profils	9 vues écrites à la main
Données affichées	Firestore production	tableaux vides, non persistés

De plus, trois fichiers `index.jsx` totalisant **1,65 Mo** contenaient des extraits bruts du monolithe mais n'étaient **importés nulle part** : les fichiers `index.js` censés les exposer ne contenaient qu'une ligne de commentaire. Ce code était donc inerte.

Conséquence — livrer cette arborescence comme « migration non-régression » aurait signifié facturer l'extraction d'une application de 68 977 lignes tout en livrant une application différente : sans les 120 écrans, sans Firestore, et avec des tableaux de bord vides. Le contrat impose l'inverse : « *sans modification de l'interface ni de la logique métier* ». Cette arborescence a donc été écartée du livrable — **sans être supprimée**, son sort restant une décision client (section 12).

À noter : le document `ARCHITECTURE_v2.md` décrit bien une trajectoire Firestore → Supabase, mais en *strangler pattern* (Firestore source de vérité, double écriture, bascule pilotée par drapeau). L'arborescence trouvée ne respectait pas non plus cette stratégie : elle lisait et écrivait Supabase directement.

3. Principe retenu — extraction mécanique plutôt que réécriture

Deux voies étaient possibles pour passer d'un fichier unique à une architecture modulaire.

Approche	Principe	Risque
Réécriture	Reconstruire chaque écran dans la nouvelle architecture	Chaque écran devient une source de régression ; aucune garantie mesurable ; volume incompatible avec le forfait
Extraction mécanique retenue	Découper le fichier d'origine par outillage, en conservant le code caractère pour caractère	Le comportement ne peut pas changer, puisque le code ne change pas ; la fidélité est vérifiable automatiquement

Pourquoi c'était réalisable ici

Une analyse structurelle préalable du monolithe a montré des conditions particulièrement favorables :

Mesure	Valeur	Lecture
Analyse syntaxique du fichier	0 erreur	fichier sain, analysable intégralement
Instructions de premier niveau	355	structure « à plat », sans imbrication
Déclarations nommées	337	chacune peut devenir un fichier
Liens de dépendance entre déclarations	702	—
Couplage moyen	2,0	chaque déclaration n'en référence que 2 autres en moyenne

Un couplage moyen de 2,0 signifie que le monolithe était, dans les faits, déjà composé de blocs largement indépendants — simplement rassemblés dans un seul fichier. Le découpage consistait donc à **matérialiser une structure déjà présente**, pas à en inventer une.

4. Comment les modules sont créés

Les 334 fichiers ne sont pas écrits à la main : ils sont **générés** par un outil livré avec le projet (`scripts/migrate/extract-modules.cjs`). Comprendre cette chaîne, c'est comprendre pourquoi le résultat est fidèle par construction.

Étape 1 — Lecture du fichier d'origine

Le monolithe est analysé par l'analyseur syntaxique Babel, le même que celui utilisé pour compiler l'application. On obtient une représentation exacte du code : chaque déclaration, sa position de début et de fin au caractère près, ses commentaires d'en-tête.

Étape 2 — Inventaire des déclarations

L'outil recense les 355 instructions de premier niveau et les classe :

- **264 fonctions** — composants d'écran, utilitaires de calcul, hooks ;
- **72 déclarations de variables** — tables de configuration, référentiels, caches ;
- **2 classes** ;
- **17 instructions à effet de bord** — code qui s'exécute au démarrage.

Étape 3 — Construction du graphe de dépendances

Pour chaque déclaration, l'outil parcourt son contenu et relève **toutes** les références aux autres déclarations. Trois formes sont détectées, et c'est ce qui distingue une extraction fiable d'une extraction approximative :

Forme de référence	Exemple	Pourquoi elle compte
Identifiant classique	<code>calculerPaieOuvrier(...)</code>	appel de fonction ou lecture de constante
Élément JSX	<code><KPICard /></code>	un composant utilisé dans l'affichage
Membre JSX	<code><ToastContext.Provider></code>	invisible pour une détection naïve — omise, l'écran plante

Les identifiants redéfinis localement sont ignorés : si une fonction déclare sa propre variable `data`, elle ne crée pas de dépendance vers une éventuelle déclaration globale du même nom.

Étape 4 — Affectation à un module métier

Chaque déclaration est rattachée à l'un des 12 modules selon une table de règles lisible et modifiable (`scripts/migrate/module-map.cjs`). Exemple : tout ce qui commence par `Qualite`, `DQR`, `PFQ` ou `Chef` rejoint le module **qualite**. Ce qui n'appartient clairement à aucun domaine métier rejoint **shared** (socle commun : graphiques, cartes KPI, panneaux, utilitaires de formatage).

Étape 5 — Découpage à l'octet près

Le texte de chaque déclaration est extrait par ses positions exactes, commentaires d'en-tête inclus. **Aucune reformulation, aucune reformatation, aucun changement de style**. C'est ce qui rend la fidélité vérifiable.

Étape 6 — Génération des imports

À partir du graphe de l'étape 3, l'outil écrit en tête de chaque fichier les `import` nécessaires, avec le chemin relatif correct. Les imports ne sont donc ni devinés ni écrits à la main : ils **découlent des références réellement observées** dans le code.

Étape 7 — Bootstrap et barrels

Les 17 instructions de démarrage sont regroupées dans `bootstrap.jsx`, **dans leur ordre d'origine**, jusqu'à l'appel final qui affiche l'application. Chaque module reçoit un fichier `index.js` listant ses exports, ce qui permet d'importer un module entier d'une seule ligne.

Conséquence pratique — `public/app.jsx` reste la source de vérité tant que la bascule n'est pas validée. `src/modules/` peut être **régénéré intégralement** à tout moment par `npm run migrate:extract`. Il n'y a donc aucun risque de divergence entre l'ancien et le nouveau code pendant la période de transition.

5. La structure produite

Les 334 fichiers sont répartis en 12 modules. Chaque fichier contient une seule déclaration : un écran, un composant, un utilitaire ou une table de configuration.

Module	Fichiers	Lignes	Écrans	Fichier le plus volumineux
shared — socle commun	73	5 948	4	DashboardTab (1 443)
agronomie	49	8 043	17	AgroAnalyseFoliairesTab (1 067)
caisse	40	3 867	1	CaisseTransactionsSub (757)
finance	34	7 192	17	FinLiquidationsTab (1 273)
qualite	26	12 125	20	QualiteLiquidationsTab (1 913)
technique	24	5 544	9	IrrigationIntelligenceTab (1 303)
rh — RH & paie	23	7 995	7	QuinzaineTab (3 153)
admin	20	4 518	8	ParametresTab (646)
achats	15	7 018	15	AchatsBonApportTab (1 762)
recolte	14	5 490	9	CoutRecolteTab (1 407)
magasin	11	2 970	9	MagReceptionTab (578)
securite	5	830	5	SecurityRegistreTab (241)
Total	334	71 540	121	+ bootstrap.jsx

Le total de lignes (71 540) dépasse légèrement les 68 977 d'origine : la différence correspond aux en-têtes de commentaire et aux lignes `import` / `export` générées dans chaque fichier. Le code métier, lui, est inchangé.

Arborescence

```

src/modules/
├─ bootstrap.jsx      point d'entrée – démarrage dans l'ordre d'origine
├─ index.js           barrel global
├─ shared/            socle : graphiques, cartes KPI, panneaux, utilitaires
│   └─ index.js
│       └─ KPICard.jsx
│           └─ Panel.jsx
│               └─ SimpleAreaChart.jsx
│                   └─ reactHooks.jsx
│                       └─ ... (73 fichiers)
├─ finance/
├─ achats/
├─ caisse/
├─ admin/
├─ qualite/
├─ agronomie/
├─ technique/
├─ securite/
├─ rh/
├─ magasin/
└─ recolte/

```

Règle d'organisation

Un fichier = une déclaration. Ce choix rend chaque écran localisable immédiatement par son nom, supprime les conflits lors de modifications simultanées, et permet à l'outil de construction de ne livrer que le code réellement utilisé.

Un seul fichier dépasse 2 000 lignes : `QuinzaineTab.jsx` (3 153 lignes). Le découper supposerait de réécrire le composant, ce que le périmètre non-régression interdit. Ce point est laissé à l'appréciation du client (section 12).

6. Anatomie d'un fichier migré

Voici un fichier réellement produit, `src/modules/qualite/QualiteBrixTab.jsx`, commenté zone par zone.

```
/* Migré depuis public/app.jsx – extraction verbatim (non-régression). ← ①
   Module: qualite | Déclaration(s): QualiteBrixTab */

import { KPICard } from '../shared/KPICard.jsx'; ← ②
import { Panel } from '../shared/Panel.jsx';
import { SimpleAreaChart } from '../shared/SimpleAreaChart.jsx';
import { cachedFetch } from '../shared/cachedFetch.jsx';
import { useState } from '../shared/reactHooks.jsx';
import { deduplicateExpeditions } from './deduplicateExpeditions.jsx'; ← ③

// ===== QUALITE BRUX TAB ===== ← ④
function QualiteBrixTab({ data, applyVarietyMapping }) {
  const [selectedFerme, setSelectedFerme] = useState('');
  ... ← ⑤
}

export { QualiteBrixTab }; ← ⑥
```

Zone	Rôle
①	En-tête de traçabilité — généré. Indique l'origine, le module d'affectation et la déclaration contenue. Permet de remonter au monolithe à tout moment.
②	Imports du socle commun — générés à partir des références réellement observées. Cet écran utilise une carte KPI, un panneau, un graphique, le cache d'appels réseau et le hook <code>useState</code> .
③	Import interne au module — chemin relatif <code>./</code> : la fonction utilitaire vit dans le même module métier.
④	Commentaire d'origine — repris tel quel depuis le monolithe. Rien n'est reformulé.
⑤	Corps du composant — identique à l'octet près . L'indentation d'origine (héritée du fichier unique) est même conservée, précisément pour que la comparaison automatique reste possible.
⑥	Export — généré, rend la déclaration importable par les autres modules.

Le fichier de démarrage

`bootstrap.jsx` rassemble les instructions qui s'exécutent au chargement, dans leur ordre d'origine : contrôle de version, interception réseau, exposition de fonctions globales pour les scripts externes, gel des référentiels, restauration du dernier profil et du dernier onglet, puis affichage de l'application :

```
ReactDOM.render(
  <ErrorBoundary><ToastProvider><WorkerDetailProvider>
    <App />
  </WorkerDetailProvider></ToastProvider></ErrorBoundary>,
  document.getElementById('root')
);
```

Cet appel est la **dernière ligne du monolithe d'origine**, reprise sans modification.

7. Cas particuliers traités

Passer d'un fichier unique à des modules change certaines règles du langage. Sept situations auraient produit des pannes silencieuses ; chacune a été détectée par les contrôles automatisés, puis traitée.

7.1 — Les hooks React déstructurés (impact : 168 fichiers)

Le monolithe déclarait en une ligne : `const { useState, useEffect, useMemo, useCallback, useRef } = React;`

Cette forme déstructurée n'était pas reconnue comme une déclaration nommée par la première version de l'outil. Résultat : **168 fichiers** utilisaient ces hooks sans les importer. **Le projet se compilait sans la moindre erreur** — seul un test en navigateur révélait la panne. Corrigé : ces cinq hooks vivent désormais dans `shared/reactHooks.jsx` et sont importés partout où ils servent.

Enseignement — « le build passe » ne prouve rien sur une migration de ce type. C'est la raison pour laquelle la validation repose sur un parcours réel en navigateur, écran par écran (section 9).

7.2 — Les contextes React en JSX

`<ToastContext.Provider>` et `<WorkerDetailContext.Provider>` sont des références d'une forme particulière (membre JSX), invisible à une détection classique. Sans traitement, les notifications et la fiche ouvrier auraient cessé de fonctionner. La détection couvre désormais cette forme.

7.3 — L'état mutable partagé

Un module ES interdit d'écrire dans une variable importée. Les 9 écritures inter-déclarations du monolithe ont été inventoriées. Sept concernaient des variables écrites uniquement au démarrage : elles ont été placées dans `bootstrap.jsx` avec leur code d'initialisation. Les deux dernières invalidaient un cache (`_bonsCache = null`) depuis deux écrans distincts : un accesseur dédié a été généré. Ce sont les **2 seuls écarts** du livrable (section 8).

7.4 — Les variables globales implicites

Dans un script classique, une fonction déclarée au premier niveau devient automatiquement accessible globalement. Ce n'est plus vrai dans un module. Les 25 scripts externes du projet (`public/lib/` et `public/components/`) ont donc été audités un par un : **aucun** ne dépend d'une variable globale implicite — les correspondances relevées sont des commentaires ou des accès explicites via `window`. Les 7 expositions volontaires (dont `window.PointageTab`) sont préservées dans `bootstrap.jsx`.

7.5 — L'ordre de démarrage

Les instructions de démarrage doivent s'exécuter dans le même ordre qu'avant : l'interception réseau doit précéder le premier appel, les référentiels doivent être remplis avant lecture. `bootstrap.jsx` préserve cet ordre à l'identique.

7.6 — Le code mort

`CampagneTab` et huit fonctions associées ne sont référencés nulle part : l'onglet réellement affiché provient d'un fichier séparé (`components/CampagneAnalytiqueTab.js`). Ce code était déjà inerte dans le monolithe. Il est extrait, puis écarté du bundle final par l'outil de construction — d'où une partie des 24,7 % de réduction de taille, **sans aucune perte de fonctionnalité**.

7.7 — L'environnement d'exécution

La page HTML de production charge une vingtaine de bibliothèques externes (React, Firebase, XLSX, jsPDF, Leaflet) et une quarantaine de scripts internes. La page livrée en est une **copie conforme** : tous ces scripts sont conservés dans le même ordre. Une seule ligne change :

```
- <script defer src="app.js?v=mt4mnskr"></script>  
+ <script type="module" src="/app.modular.js"></script>
```

8. Vérification de fidélité

La fidélité n'est pas déclarée : elle est **mesurée**. La commande `npm run migrate:verify` ré-analyse les 334 fichiers produits, isole le corps de chaque déclaration et le recherche caractère pour caractère dans le monolithe d'origine.

```
$ npm run migrate:verify

top-level statements checked : 355      ← identique au monolithe
byte-identical to app.jsx    : 353
generated setters (expected) : 1
NON-VERBATIM                : 2
files with UNRESOLVED refs   : 0
parse failures               : 0 / 348 fichiers
```

Les 3 écarts, un par un

#	Emplacement	Avant → Après	Motif
1	<code>app.jsx:18358</code> qualite/QualiteProductionTab	<code>_bonsCache = null</code> → <code>__set_bonsCache(null)</code>	Un module ES interdit d'affecter une variable importée
2	<code>app.jsx:55043</code> achats/AchatsBonApportTab	<code>_bonsCache = null</code> → <code>__set_bonsCache(null)</code>	Identique
3	<code>shared/loadBonsFromFirestore</code>	ajout de <code>export function</code> <code>__set_bonsCache(v){ _bonsCache = v; }</code>	Accesseur généré pour les deux appels ci-dessus

Les deux appels modifiés sont des **invalidations de cache** : ils remettent une variable à `null` pour forcer un rechargement. Passer par un accesseur produit **exactement le même effet**. Aucun calcul, aucun montant, aucun affichage n'est concerné.

Autrement dit : sur 68 977 lignes, **2 lignes** ont été réécrites, toutes deux mécaniquement équivalentes, toutes deux documentées et localisables.

Contrôles complémentaires

Contrôle	Résultat
Références croisées non résolues entre modules	0
Fichiers en échec d'analyse syntaxique	0 sur 348
Déclarations exportées	337 — la totalité
Scripts externes dépendant d'une globale implicite	0 sur 25 audités

9. Vérification fonctionnelle

La fidélité du code ne suffit pas : encore faut-il que l'application se comporte à l'identique. Un banc de test automatisé pilote un navigateur réel, sélectionne chaque profil, ouvre chaque écran, et compare les deux versions servies côte à côte.

Protocole

1. Les deux versions sont servies simultanément : l'ancienne (`legacy.html` , monolithe) et la nouvelle (`index.html` , 334 modules).
2. Pour chacune : les **20 profils** sont sélectionnés l'un après l'autre.
3. Pour chaque profil : **chaque entrée de menu est réellement cliquée**.
4. Après chaque rendu, on relève les erreurs console, l'état d'erreur éventuel de l'écran, et l'empreinte du contenu affiché.
5. Les deux relevés sont comparés paire par paire.

Résultat

Mesure	Monolithe <code>legacy.html</code>	Modulaire <code>index.html</code>
Profils parcourus	20	20
Écrans ouverts	350	350
Écrans rendus correctement	350	350
Écrans en erreur	0	0
Écrans « Erreur chargement »	0	0

Comparatif détaillé

Comparaison sur 345 paires (profil × écran)	Résultat
Écrans présents d'un seul côté	0
Différences d'erreurs console	0
Différences de contenu affiché > 2 %	1 — voir ci-dessous

L'unique écart concerne l'écran **Suivi Modifications**. Il a été mesuré sur trois exécutions consécutives de **chaque** version : l'ancienne affiche 33, 43 puis 33 lignes ; la nouvelle 32, 29 puis 38. Cet écran lit un journal d'audit en temps réel, dont le contenu varie d'une seconde à l'autre. **La variation est donc inhérente à l'écran, pas à la migration**, et présente à l'identique dans les deux versions.

L'écran de connexion a par ailleurs été comparé octet par octet : **3 875 octets de contenu affiché des deux côtés**, aucune erreur.

Conclusion — sur 350 écrans, 20 profils et 345 paires comparées, aucune différence de comportement n'a été relevée entre le monolithe et la version modulaire.

10. Déploiement de préversion

Une préversion est en ligne pour permettre la recette par l'équipe, sans toucher à la production.

Page	Contenu
/	Interface d'origine, servie par le build modulaire (334 modules) — le livrable
/legacy.html	Monolithe <code>app.js</code> — conservé comme référence de comparaison

Ce qui n'est pas touché

- **L'hébergement Firebase de production reste inchangé** : sa configuration (`firebase.json`) sert toujours `public/` et reconstruit `app.js` depuis `app.jsx`. Aucun déploiement de production n'a été déclenché.
- Le backend (Cloud Functions, région `europa-west1`) est inchangé. La préversion n'héberge que le frontend ; les **54 routes** `/api/*` sont redirigées vers les mêmes fonctions que la production.
- `public/app.jsx`, `public/lib/` et `public/components/` sont inchangés.

Mode démonstration

La préversion s'ouvre **sans authentification**, afin que l'équipe puisse parcourir l'application sans compte. Ce contournement réutilise un mécanisme déjà présent dans le projet, **patché uniquement dans la sortie de build** : les fichiers sources restent intacts, l'hébergement de production ne peut donc pas l'embarquer par accident.

Aucune donnée réelle n'est exposée. Les règles Firestore refusent toute lecture non authentifiée : la règle par défaut est `allow read, write: if false` et les 59 règles de lecture exigent toutes une authentification. Aucune règle permissive n'existe. L'équipe voit donc la **structure** de l'application — écrans, navigation, 20 profils — et non les chiffres d'exploitation.

Rétablir l'écran de connexion se fait en une ligne de configuration, sans modification de code.

Correctif du harnais de démonstration

Le mécanisme de démonstration répondait « succès » à toute route `/api/*` non simulée, mais avec une réponse **vide**. L'application enregistrait alors des objets dépourvus de leurs champs, puis les lisait en profondeur — d'où plusieurs écrans en erreur, **dans les deux versions**. Le correctif fournit des réponses **bien formées** : listes vides, objets vides et valeurs à zéro. Zéro signifie « pas de donnée », ce n'est pas un chiffre inventé.

Écran	Champ attendu	Valeur fournie
Dashboard Pointage	liste des nouveaux ouvriers	liste vide
CPC / Dashboard	total de la quinzaine	0
Carburant	prix moyen au litre	0
Maroc Télécom	coût moyen par ligne	0
Dashboard Quinzaine	répartition par ferme	liste vide
Prédiction Récolte	prévision du jour	objet à zéro
Pointage Divers	entrées de la journée	objet vide

Ce correctif ne touche **que la sortie de démonstration** : les sources restent inchangées et la fidélité 353/355 est préservée.

11. Ce qui n'a pas changé

Cette section est la contrepartie de toutes les précédentes : elle énumère ce qui a été délibérément laissé intact.

Élément	État
Interface utilisateur	Identique — aucun écran, libellé, couleur ou disposition modifié
Logique métier	Identique — paie, réconciliations, calculs de coûts, workflows de validation
Parcours utilisateur	Identique — 20 profils, mêmes menus, mêmes droits
Base de données	Firestore, inchangée — aucune migration de données
Backend (Cloud Functions)	Inchangé — aucune fonction modifiée ni redéployée
Règles de sécurité	Inchangées
Hébergement de production	Inchangé — aucun déploiement déclenché
<code>public/app.jsx</code>	Inchangé — reste la source de vérité
Bibliothèques externes et scripts internes	Inchangés, chargés dans le même ordre

Réversibilité

Le retour à l'état antérieur ne demande aucune manipulation de code : il suffit de servir `app.js` au lieu de `app.modular.js` — soit une ligne dans la page HTML. Le monolithe n'ayant jamais été modifié, il reste fonctionnel en permanence.

12. Points ouverts

Cette section liste, sans omission, ce qui n'est pas couvert par la présente livraison.

12.1 — Objectifs du plan non couverts

Objectif (plan v1.1)	État	Commentaire
≥ 11 modules, fichiers < 2 000 lignes	Fait	12 modules ; seul <code>QuinzaineTab</code> (3 153) dépasse
Non-régression 100 % des pages	Fait	350 écrans, 20 profils, parité complète
Code splitting, chargement < 500 Ko	Non fait	Exige un chargement différé qui modifie la temporalité de rendu — à valider avant engagement
Lighthouse > 90	Non mesuré	Dépend du point précédent
Tests unitaires (300+, > 60 %)	Non fait	Vitest n'est pas installé
TypeScript progressif	Non fait	—
Dual mode / feature flags	Non câblé	Le drapeau existe mais ne pilote aucune bascule
Tests E2E Playwright (5-10 parcours)	Partiel	350 rendus couverts ; pas de parcours métier complet
Déploiement prod + monitoring Sentry	Non fait	—

12.2 — Décisions attendues du client

Sujet	Décision attendue
Arborescence <code>src/features/</code>	La réécriture Supabase (section 2) coexiste avec la migration. Elle n'a pas été supprimée. Faut-il l'archiver, la retirer, ou la reprendre dans un chantier distinct ?
Découpage de <code>QuinzaineTab</code>	3 153 lignes. Le découper suppose de réécrire le composant : hors périmètre non-régression, à traiter en régie.
Bascule de production	Passer l'hébergement Firebase sur le build modulaire, après recette.

12.3 — Défauts préexistants relevés

La migration a mis en lumière des fragilités **déjà présentes** dans l'application. Elles ne sont pas causées par les travaux et n'ont pas été corrigées, toute modification sortant du forfait non-régression.

Défaut	Description
Lectures non protégées	Plusieurs écrans lisent des champs imbriqués sans vérification préalable (<code>nouveauxData.workers</code> , <code>carb.prixMoyenLitre</code> , <code>prediction.today</code> ...). Si l'API renvoie un succès avec une réponse incomplète, l'écran tombe en erreur. Une protection d'une ligne par cas suffirait.
Clé API exposée	Une clé OpenWeather figurait en clair dans le code et dans un message de commit . Elle a été sortie vers les variables d'environnement et purgée de l'historique de la branche. Cette clé doit néanmoins être renouvelée : une clé ayant circulé doit être considérée comme compromise.

Action requise côté client — renouveler la clé OpenWeather sur `openweathermap.org`, puis la renseigner dans les variables d'environnement du projet.

13. Reproductibilité

Aucune étape n'est manuelle : l'ensemble de la chaîne est livré sous forme de scripts exécutables, versionnés dans le dépôt.

Commandes

Commande	Effet
<code>npm run migrate:extract</code>	Régénère intégralement <code>src/modules/</code> depuis <code>public/app.jsx</code>
<code>npm run migrate:verify</code>	Vérifie la fidélité à l'octet et l'absence de référence non résolue
<code>npm run migrate:build</code>	Produit le bundle modulaire
<code>npm run migrate:walk</code>	Parcourt l'application en navigateur, écran par écran
<code>npm run migrate:all</code>	Enchaîne extraction, vérification et build

Outils livrés — `scripts/migrate/`

Fichier	Rôle
<code>extract-modules.cjs</code>	Le générateur : analyse, graphe de dépendances, découpage, écriture des imports
<code>module-map.cjs</code>	Table d'affectation des déclarations aux 12 modules — lisible et modifiable
<code>verify-fidelity.cjs</code>	Comparaison à l'octet avec le monolithe
<code>verify-references.cjs</code>	Détection des références non résolues
<code>verify-globals.cjs</code>	Audit des dépendances aux variables globales implicites
<code>walk-tabs.cjs</code>	Banc de test navigateur : profils, écrans, comparaison

Propriété importante — l'extraction est **idempotente** : relancée, elle produit exactement le même résultat. Si `public/app.jsx` évolue pendant la période de transition, une seule commande suffit à resynchroniser les 334 modules. Il n'y a donc aucun risque de divergence entre les deux versions.

Annexe A — Journal des livraisons

Les travaux ont été livrés en 14 incréments traçables, chacun documenté et vérifié. Branche : `main` · Dépôt : `CElyousfi/cloneBERRYGOOD`.

Réf.	Objet
<code>2a611de</code>	Extraction non-régression du monolithe vers <code>src/modules</code> (334 fichiers)
<code>4a3071a</code>	Sécurité : clé OpenWeather sortie vers les variables d'environnement
<code>10c4dd4</code>	Build : suppression du téléchargement inutile des navigateurs de test
<code>f6e2f6d</code>	Préversion : mode démonstration sans authentification
<code>c4555df</code>	Intégration de la migration dans <code>main</code>
<code>96ffa4e</code>	Correctif de configuration de déploiement
<code>0605c49</code> → <code>8d6921c</code>	Explorations d'interface (thèmes et coquilles) — retirées du livrable
<code>780f1dc</code>	Vérification étendue : 413 écrans parcourus
<code>d645a91</code>	Recentrage sur le livrable : interface d'origine sur build modulaire
<code>c1af533</code>	Nettoyage du suivi de fichiers de travail
<code>b5281fc</code>	Correctif du harnais de démonstration — 350/350 écrans validés

Les explorations d'interface (nouveaux thèmes, refonte visuelle) ont été menées à la demande puis retirées du livrable, le client ayant confirmé vouloir conserver l'interface d'origine. Elles restent consultables dans l'historique du dépôt.

Documentation technique associée

`MIGRATION_REPORT.md`, à la racine du dépôt, contient le détail technique complet : commandes, sorties de vérification brutes, journal des cas particuliers et décisions d'architecture.

Document généré à partir des données réelles du dépôt (comptages de fichiers, sorties de vérification, relevés du banc de test navigateur). Aucun chiffre n'y est estimé.